PROJECT VIVA PREPARATION GUIDE

ElevAI — Most Expected Viva Questions & Simple Answers

Easy-to-explain answers covering Overall Architecture, Core Features, AI, and Practical Concepts

PART 1: GENERAL PROJECT OVERVIEW & BASICS

High-level questions about the project purpose, users, and problem statement

Q1. What is ElevAI and what problem does it solve?

ElevAI is an AI-powered smart career preparation and job recruitment web portal. It combines mock interview practice, resume scoring, and job applications into one single platform.

- The Problem: Job seekers usually practice mock interviews on one site, check resume scores on another site, and search for jobs on third-party job boards. Recruiters also have no easy way to know if an applicant has actually practiced or has good communication skills.
- The Solution: ElevAI connects candidate preparation directly to hiring. Candidates practice AI interviews and optimize resumes, and recruiters can view those verified scores when reviewing job applications.

HOW TO ANSWER IN 1 LINE: "ElevAI is a unified platform that helps candidates practice AI mock interviews, test resume ATS scores, and apply for jobs while helping recruiters screen candidates using verified AI scores."

Q2. Who are the users of this application?

There are two main types of users:

- 1. Candidates (Job Seekers / Students): They take AI voice mock interviews, upload resumes for instant ATS feedback, search for jobs, apply with one click, and track their application status.
- 2. Admins / Recruiters: They create job openings, review applicant profiles (including their resume and mock interview scores), shortlist candidates, and schedule real interviews.

HOW TO ANSWER IN 1 LINE: "We have two roles: Candidates who prepare and apply, and Admins/Recruiters who post jobs and review applicant dossiers."

Q3. How is ElevAI different from platforms like LinkedIn or LeetCode?

ElevAI offers an all-in-one closed loop:

- LeetCode only tests coding questions; it does not provide conversational speech interviews or resume scoring.
- LinkedIn allows job applications, but does not offer built-in AI mock interviews with automated 4-metric scorecards.
- ElevAI brings preparation (AI interview + ATS resume scoring) and job application together in one place.

Q4. Can you explain the complete workflow for a Candidate from start to finish?

A candidate goes through 5 simple steps:

- Step 1: Sign up and log in to the portal.
- Step 2: Go to Mock Interview -> Select a domain (e.g., Fullstack) and difficulty -> Answer questions using voice or text -> Get instant AI score and feedback.
- Step 3: Go to Resume Vault -> Upload PDF resume -> View ATS score and missing skill keywords.
- Step 4: Go to Job Board -> Browse openings -> Click "Apply" and attach their resume.
- Step 5: Check notifications and Track Applications page to see if they are Shortlisted or Scheduled for an interview.

HOW TO ANSWER IN 1 LINE: "Register -> Practice AI Interview -> Check Resume ATS Score -> Apply for Jobs -> Track Application Status."

PART 2: TECHNOLOGY STACK & ARCHITECTURE

Simple questions explaining the technologies used and why you chose them

Q5. What is the overall Technology Stack of your project?

We used a modern full-stack web architecture:

- Frontend: React.js (built with Vite) and Tailwind CSS for a fast, responsive Single Page Application (SPA).
- Backend: Node.js and Express.js for building the REST API server.
- Database: PostgreSQL hosted on Supabase for relational data storage (users, jobs, applications).
- File Storage: Supabase Storage Bucket for storing uploaded resume PDF files.
- Artificial Intelligence: Google Gemini 1.5 Flash API for generating questions and grading answers.
- Authentication: JSON Web Tokens (JWT) and Bcrypt for secure password hashing.

HOW TO ANSWER IN 1 LINE: "Our stack is React and Tailwind on the frontend, Node.js and Express on the backend, PostgreSQL and Supabase for database and storage, and Google Gemini AI for intelligence."

Q6. What is the architecture of your system?

Our project follows a standard Client-Server REST API architecture:

- Client (Frontend): The React app runs in the user's browser and handles the user interface and state.
- Server (Backend): The Express.js server receives HTTP requests, verifies authentication, executes business logic, and communicates with the database and AI.
- Database & AI Services: Supabase stores relational records and files, while Google Gemini processes intelligent text prompts.

HOW TO ANSWER IN 1 LINE: "It is a 3-tier Client-Server architecture: React frontend communicates with an Express REST API backend, which connects to a PostgreSQL database and Google Gemini AI."

Q7. Why did you choose React.js instead of simple HTML, CSS, and JavaScript?

React makes the website much faster and smoother for the user:

- Single Page Application (SPA): Pages switch instantly without refreshing or reloading the browser.
- Reusable Components: UI elements like the navbar, job cards, modals, and timer buttons are built once and reused everywhere.
- Smooth State Management: It makes real-time features like the countdown timer and live speech-to-text recording seamless.

Q8. Why did you use PostgreSQL (Supabase) instead of MongoDB?

Because our project data is strictly relational:

- Clear Relationships: An Application connects a User, a Job, and a Resume. Relational databases enforce these links with Foreign Keys.
- Data Integrity: If an admin deletes a job, all applications for that job are cleanly handled using Cascade Deletes.
- Supabase provides managed PostgreSQL with built-in cloud storage buckets for resumes, saving extra server setup.

PART 3: CORE FEATURES & WORKING MECHANISM

Easy explanations of how the main features actually operate

Q9. How does the AI Mock Interview feature work?

The mock interview works in 3 clear steps:

- 1. Question Generation: The candidate chooses a domain and difficulty. The backend asks Gemini AI to generate technical interview questions.
- 2. Answering with Voice/Timer: The system reads questions aloud (Text-to-Speech). The candidate speaks their answer into their microphone, which is transcribed into text in real time.
- 3. AI Evaluation: When submitted, Gemini AI grades the answers and returns an Overall Score (0-100) plus 4 sub-scores: Domain Knowledge, Clarity, Confidence, and Communication, along with improvement tips.

HOW TO ANSWER IN 1 LINE: "Gemini generates questions based on role, the user answers via voice or text with a live timer, and Gemini grades the answers with a 4-metric scorecard."

Q10. How does the Voice / Speech feature work without paying for external audio APIs?

We use the browser's built-in Web Speech API:

- Speech-to-Text: The browser's native SpeechRecognition listens to the candidate's microphone and converts their spoken words into text.
- Text-to-Speech: The browser's native window.speechSynthesis reads the AI questions aloud with natural voice output.
- Benefit: It works directly inside modern browsers (Chrome, Edge) with zero latency and zero extra API cost.

HOW TO ANSWER IN 1 LINE: "We use the native browser Web Speech API for both listening (Speech-to-Text) and speaking questions aloud (Text-to-Speech)."

Q11. How does the Resume ATS Scorer work?

When a candidate uploads a PDF resume:

- 1. Text Extraction: The backend uses a library called pdf-parse to read all text from the PDF file.
- 2. AI ATS Analysis: The text is sent to Gemini AI, which checks for formatting, action verbs, and technical keywords.
- 3. Feedback Display: The system displays an ATS Score (0-100), key strengths, areas to improve, and a list of missing skill keywords.

HOW TO ANSWER IN 1 LINE: "The backend extracts text from the PDF using pdf-parse, sends it to Gemini AI, and displays an ATS score with missing keywords and improvement tips."

Q12. What are the 6 stages of a Job Application in your portal?

ElevAI tracks every application through a 6-stage lifecycle:

- 1. Applied: Candidate submitted the application.
- 2. Reviewed: Recruiter opened and reviewed the application.
- 3. Shortlisted: Candidate passed preliminary screening.
- 4. Interview Scheduled: Recruiter scheduled a live interview with date and meeting link.
- 5. Accepted: Candidate received a final job offer.
- 6. Rejected: Application was declined at any stage.

HOW TO ANSWER IN 1 LINE: "Applied -> Reviewed -> Shortlisted -> Interview Scheduled -> Accepted or Rejected."

Q13. What is the Admin Review Dossier?

In the Admin Panel, when a recruiter clicks on an applicant:

- It opens a complete candidate profile showing their contact details, uploaded resume, and verified AI mock interview scores.
- This allows the recruiter to see both the resume AND the candidate's interview communication scores before scheduling a real interview.

PART 4: SECURITY, LOGIN & AUTHENTICATION

Simple questions on how user security and roles are handled

Q14. How does User Login and Authentication work in your app?

We use JSON Web Tokens (JWT) for secure, stateless authentication:

- 1. Login: The user enters email and password. The backend checks if the password matches the hashed password in the database.
- 2. Token Generation: If valid, the backend creates a signed JWT token containing the user's ID and Role (candidate or admin).
- 3. Token Storage: The frontend stores this token and sends it in the request header (Authorization: Bearer <token>) for every action.

HOW TO ANSWER IN 1 LINE: "We use JWT tokens. When the user logs in, a token is issued to the browser and sent in headers to verify who is making each request."

Q15. How are user passwords stored securely in the database?

Passwords are never stored as plain text. We use the bcrypt library to hash passwords with 10 salt rounds before saving them in PostgreSQL. This means even if someone views the database, they cannot see the actual passwords.

HOW TO ANSWER IN 1 LINE: "Passwords are encrypted using bcrypt with salt rounds, so plain-text passwords are never saved in the database."

Q16. How do you prevent a normal candidate from accessing Admin pages or APIs?

We use Role-Based Access Control (RBAC):

- On the Frontend: React checks user.role. If a candidate tries to visit /admin, they are redirected to the candidate dashboard.
- On the Backend: A middleware function (requireAdmin) checks the user's role inside the verified JWT token. If it is not "admin", it blocks the request with HTTP 403 Forbidden.

HOW TO ANSWER IN 1 LINE: "We use role-based middleware that checks the role in the JWT token on both frontend route guards and backend API endpoints."

PART 5: PRACTICAL VIVA QUESTIONS & FUTURE SCOPE

Questions about challenges, testing, limitations, and future enhancements

Q17. What challenges did you face while developing this project and how did you solve them?

Two main engineering challenges:

- 1. Unpredictable AI formatting: Sometimes the AI returned explanations or markdown backticks instead of pure JSON. We solved this by writing a regex cleaning function in the backend to sanitize the AI response before parsing.
- 2. Large PDF processing: Uploading and parsing resumes directly on disk was slow. We solved this by using in-memory buffers (Multer memoryStorage) and streaming files directly to Supabase Storage.

Q18. How did you test your application?

We conducted comprehensive testing:

- Unit & Route Testing: Tested individual API endpoints (Auth, Resume Upload, Interview Submission, Jobs) with valid and invalid inputs.
- Authentication & RBAC Testing: Verified that protected routes reject unauthorized requests and that candidate accounts cannot access admin APIs.
- End-to-End User Journey Testing: Tested the complete flow from signing up, taking an interview, uploading a resume, applying for a job, and updating status from the admin panel.

Q19. What are the limitations of your project and what can be added in the future?

Current limitations and roadmap:

- Current Limitations: The coding round evaluates answers as text rather than running live code execution, and interview evaluation is audio/text based without video emotion tracking.
- Future Scope: (1) Integrating an in-browser live code editor (Monaco Editor) with automatic code compilation, (2) Adding WebRTC video analysis for body language and eye-contact feedback, and (3) Multi-panel HR and System Design interview simulations.

HOW TO ANSWER IN 1 LINE: "In the future, we plan to add live sandboxed coding execution and video-based body language emotion analysis."

PART 6: 1-MINUTE QUICK VIVA SUMMARY

Quick-fire answers for instant memory recall

Project Type: All-in-One AI Career Preparation and Smart Recruitment Portal.

Frontend: React.js (Vite) + Tailwind CSS (Single Page Application).

Backend: Node.js + Express.js (Modular REST API).

Database: Supabase PostgreSQL (7 Relational Tables) with Foreign Keys.

File Storage: Supabase Object Storage Bucket for Resume PDFs.

AI Engine: Google Gemini 1.5 Flash for question generation and scorecards.

Voice Audio: Native Web Speech API (SpeechRecognition and SpeechSynthesis).

Security: JWT Bearer Authentication + Bcrypt password hashing + RBAC middleware.

Application Statuses: 6 Stages: applied -> reviewed -> shortlisted -> interview_scheduled -> accepted / rejected.